

ShopEasy Term Project — Reflection Report

Course: Software Testing

Author: Anıl Aydın

Repository: Software-Testing-Project (branch `main`)

Tooling: Java 17, JUnit 5, AssertJ, Mockito, jqwik, JaCoCo, Maven Surefire (with `-ea`)

Final suite: 93 tests, all passing; total runtime under five seconds on a developer laptop.

1. Overview

The aim of this project was to apply five different testing strategies to a small Java e-commerce library called ShopEasy and to reason critically about what each strategy actually bought us. The final suite contains 93 passing tests spread across five files: 37 domain-partition tests for `PriceCalculator`, 20 structural tests for `ShoppingCart` (with JaCoCo branch coverage above the 80% target), 24 contract tests that exercise the `assert`-based pre- and post-conditions added under Task 3, six Mockito-based unit tests that drive `OrderProcessor` in isolation, and six jqwik property-based tests that probe invariants over thousands of randomized inputs. What follows is an honest account of what the suite caught, what it cost, where it sits in the testing pyramid, and what should logically come next.

2. Bugs Revealed by Testing

The most instructive defects in this project were not the ones we expected to find. Two in particular emerged only because property-based testing was layered on top of the contract assertions, and together they illustrate why combining techniques matters more than perfecting any one of them.

The first was a **commutativity defect** in `ShoppingCart.addItem`. The property under test was straightforward: the order in which two distinct products are added to a cart should not change the final total. After roughly 178 randomized trials, jqwik shrank a failure down to a remarkably small counterexample — two `Product` instances sharing the same identifier "P-000" and the same display name "AA", but with different unit prices (0.02 and 0.01) and a quantity of one each. Adding the cheaper product first yielded a total of 0.02, while adding the more expensive product first yielded 0.04. Inspecting the production code clarified the root cause: when `addItem` encounters an existing line whose product ID matches the incoming product, it merges the quantities but silently keeps the *original* `Product` reference, including its original price. The newer price is dropped on the floor. This is order-dependent state, and no example-based test we had written would have surfaced it, because no developer writes test data with two product objects that happen to share an identifier yet disagree about price. We resolved the issue at the property level by adding `Assume.that(!product1.getId().equals(product2.getId()))`, which restricts the property to its actually intended domain (distinct products) and records the limitation in the suite itself.

The second defect was subtler, and lived at the boundary between Task 3 and Task 4. The property `cartTotalIsAlwaysNonNegative` generates a discount rate in `[0.0, 100.0]` and applies it to a non-empty cart. When the discount happened to land on exactly 100.0 and the raw total was a value that cannot be represented precisely in binary floating point (for example, a cart total of 0.03),

the expression `rawTotal - (rawTotal * 100.0 / 100)` evaluated to roughly -1.4×10^{-17} instead of the mathematically correct zero. This silently violated the contract assertion `assert discounted >= 0` that had been added in Task 3, and surfaced as a hard `AssertionError` thrown from production code at runtime. The proper engineering response was not to narrow the test domain — that would have hidden the bug rather than fix it — but to relax the contract by an epsilon: `assert discounted >= -1e-9`, with the symmetric `+1e-9` applied to the upper-bound assertion. The amended contract is a faithful encoding of its actual intent (*non-negative up to numerical precision*) and is now resilient to any input the property generator can produce. The fix is a single line, but the bug class it covers is general: any contract that combines IEEE 754 arithmetic with a strict inequality is a candidate for the same treatment.

Beyond these two, the structural pass exposed an absent guard against zero or negative quantities in `updateQuantity` — later tightened by the Task 3 contracts — and the domain-partition tests on `PriceCalculator` confirmed identity behavior at the boundaries (zero discount, zero tax) that example-based testing alone tends to skip.

3. Cost-Effectiveness of the Techniques

If we rank the techniques by defects found per hour invested, property-based testing and design by contract were by some margin the most cost-effective. The two bugs described above were each discovered by a single property — fewer than thirty lines of test code — that together exercised on the order of two thousand input combinations. Reproducing that coverage with example tests would have required dozens of carefully chosen cases, and the most damaging case (two `Product` objects with the same ID but different prices) is one no reasonable developer would have written by hand.

Design by contract was nearly free at the point of use. The pre-conditions, post-conditions, and invariants are short, sit next to the code they protect, and serve both as runtime checks (under `-ea`) and as executable documentation. They acted as a force multiplier for the property tests: jqwik did not need to know what the right answer was, because the contract spoke up whenever production code reached an inconsistent state. The floating-point bug was, in effect, a contract violation discovered by random search, and that combination is difficult to beat in terms of mean time to defect.

Structural testing and example-based partition testing were valuable but more expensive. Reaching 80% branch coverage on `ShoppingCart` required twenty carefully constructed tests, several of which exercise the same logical pathway in slightly different ways. Their value is real — they pin the current behavior of the cart against future regression — but they were not the source of the most surprising defects we found. Mockito-based unit testing of `OrderProcessor` was a third category: cheap to write, very fast, and indispensable for asserting the *negative* case (`verify(paymentGateway, never()).charge(...)`), but it tells us nothing about whether the real inventory and payment integrations work, only that the orchestration logic above them is correct.

4. Position in the Testing Pyramid and What Is Missing

Every test we have written sits in the bottom layer of the testing pyramid. The full suite of 93 tests executes in just over four seconds, requires no network, no database, no filesystem state, and no external process. `OrderProcessor`, the only class with non-trivial external dependencies, is

exercised exclusively through Mockito doubles of `InventoryService` and `PaymentGateway`. This is by design — unit tests should be cheap and deterministic — but it leaves two important categories of defect entirely uncovered.

The first gap is **integration testing**. Mocking out the inventory service means we have never observed what happens when a real implementation of `isAvailable` consults a real data store while another transaction is mutating that store, when the connection drops mid-call, or when the persistence model and the in-memory `Product` class disagree about how a price is rounded. The mock answers `true` or `false` instantly and never lies; a real service does neither. A reasonable next step is to introduce an embedded database such as H2 and an in-process fake of the payment gateway, then re-run `OrderProcessor` end-to-end against that stack.

The second gap is **end-to-end and concurrency testing**. ShopEasy is, in effect, a small piece of an e-commerce system, and the properties most likely to fail in production — that two simultaneous checkouts do not oversell stock, that an order is never created without a successful charge, that the system degrades gracefully when the payment gateway is slow rather than absent — cannot be expressed at the unit level. They require a multi-threaded harness driving `OrderProcessor` from several threads against a shared, real inventory.

5. Where We Would Go Next

Given a further week of work, the first investment would be the integration layer described above: a lightweight embedded database, a fake gateway that can be configured to fail, time out, or succeed slowly, and a small set of scenario tests that walk a customer from cart to confirmed order through the full stack. The second investment would be concurrency stress on `ShoppingCart` and `OrderProcessor`, ideally driven by jqwik's action-based stateful testing so that the same property-based machinery that found the commutativity bug can also search the interleaving space. Finally, the floating-point experience in Task 3 suggests it would be worth replacing the `double`-based price arithmetic with `BigDecimal`. That is a larger refactor, but it would eliminate a whole class of subtle defects rather than treating each one with a local epsilon.

The five tasks taken together demonstrate that no single technique is sufficient, and that the most interesting bugs tend to live at the seams between techniques — a property test that exercises a contract assertion, a structural test that reveals an unguarded branch, a Mockito test that confirms a behavior by its absence. The remaining work is not to do more of any one of these, but to extend the same disciplined combination of methods into the layers above the unit boundary.